DRAM SEMICONDUCTOR · MLOPS

40 Days of DRAM Production Drift Detection

200,000,000 rows processed

1 retrain · 1 canary failure · 1 automatic rollback

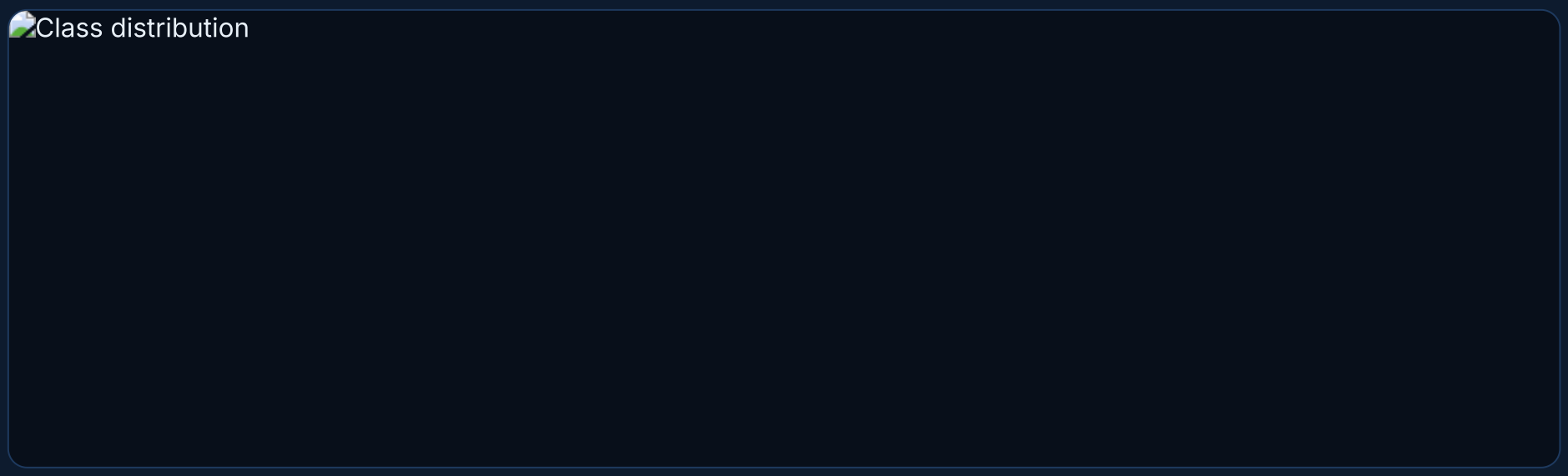
Zero human intervention

HybridTransformerCNN · Airflow · Kafka · Spark · MLflow · K8s · Prometheus

1:160	16M	317K
Class imbalance	Training records	Model parameters

THE PROBLEM

1:160 Imbalance: 99.4% accuracy predicts nothing



Left: 49,686 pass vs 314 fail in a batch (1:158). **Right:** Label noise with 42 Pass-as-Fail and 17 Fail-as-Pass noisy labels. Standard cross-entropy predicts "pass" for every chip: 99.4% accuracy, 0% defect recall.

Why FocalLoss, not SMOTE?

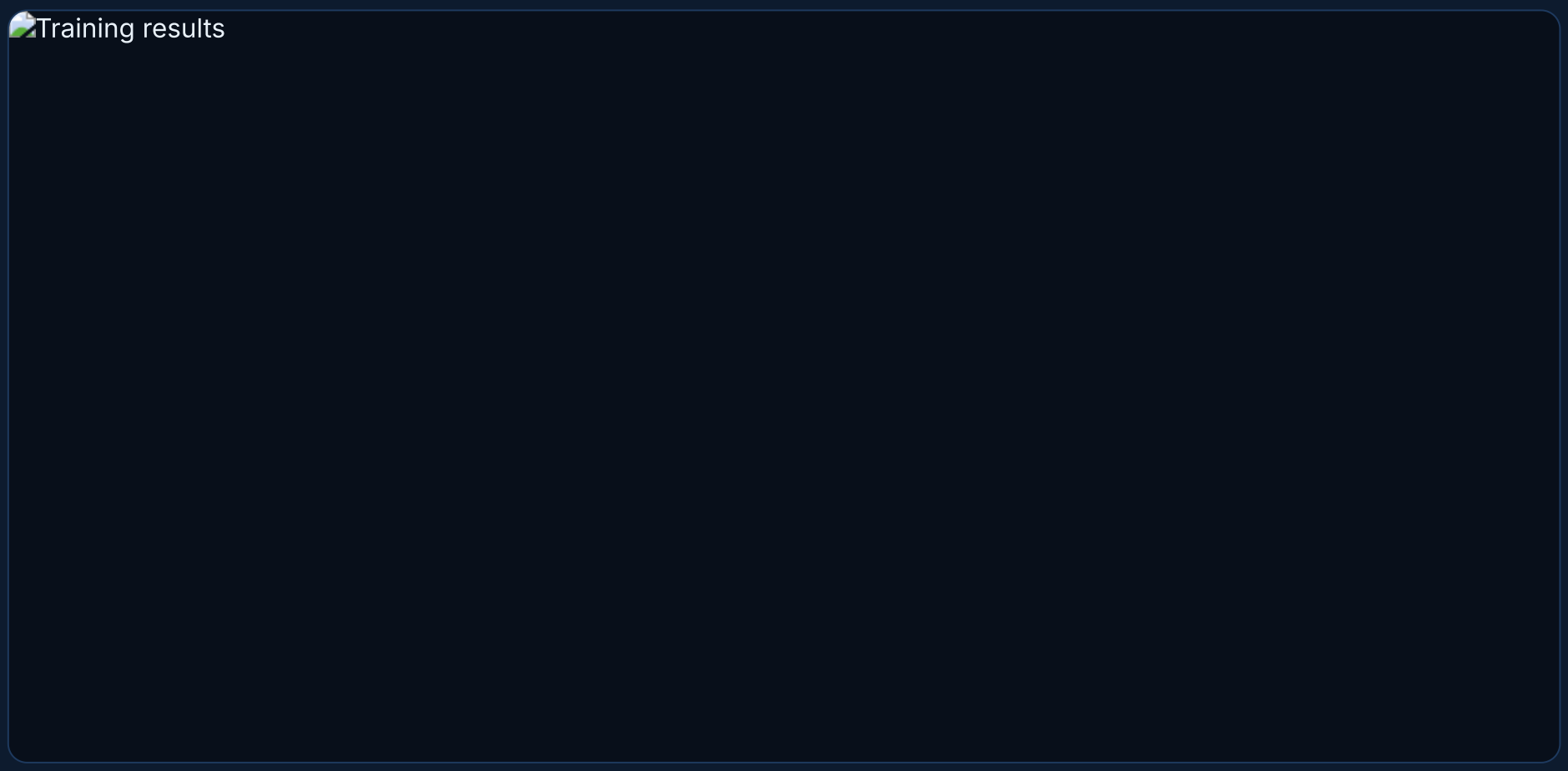
FocalLoss (α=0.75, γ=2.0) adds a modulating factor $(1-p)^{\gamma}$ to cross-entropy loss. Easy majority samples get near-zero loss weight, forcing the model to focus gradient updates on missed defective chips. **SMOTE** creates synthetic minority samples by interpolation, but for sensor data like cell leakage and retention time, interpolated values create physically impossible feature combinations. FocalLoss modifies the loss function directly without generating fake data.

DRAM Test Features

Feature	What it measures	Why it matters
`test_temp_c`	Chip temperature during test	Hot spots = process defects
`cell_leakage_fa`	Cell current leak (femtoamps)	High leak = retention failure
`retention_time_ms`	Charge hold time (ms)	Below spec = bit errors
`gate_oxide_thickness_a`	Oxide layer (Ångstroms)	Too thin = breakdown
`vt_shift_mv`	Threshold voltage shift (mV)	Off-target = unreliable
`trcd_ns`	Row-to-column delay (ns)	Slow access = timing fail

A100 GPU TRAINING

HybridTransformerCNN on NVIDIA A100-SXM4-40GB



Top left: Loss convergence over 50 epochs with bfloat16. **Top center:** AUC-PR per epoch. **Top right:** Precision-recall curves for val/test/unseen splits (near-identical = no overfitting). **Bottom:** Confusion matrices. 3,374 true defects caught out of 15,595 total. 34,075 good chips flagged for retest (acceptable: retest costs $0.10/chip vs $1000+ field failure).

0.816	0.054	50
AUC-ROC	AUC-PR (primary metric)	Epochs (best: 48)

Why AUC-PR, not AUC-ROC? At 1:160 imbalance, AUC-ROC inflates from massive true negatives. We catch 21.6% of defects but AUC-ROC shows 0.816. AUC-PR ignores true negatives and honestly reflects minority class detection. Our 0.054 is 8.6x better than random (0.00625). **A100-SXM4-40GB** is the same GPU on Colab Pro, server-grade NVLink socket with 40GB HBM2e memory.

CRITICAL ENGINEERING INSIGHT

The float16 Death Spiral that killed 4 training runs

float16 + GradScaler

```
FocalLoss: (1-p)^gamma
# 5-bit exp overflows
GradScaler / 2 / 2 / 2
scale to 0, loss collapses
```

bfloat16, no GradScaler

float16	5-bit exp = +/-65,504
bfloat16	8-bit exp = +/-3.4×10^38

bfloat16 has float32 exponent range.
Focal gradients never overflow.

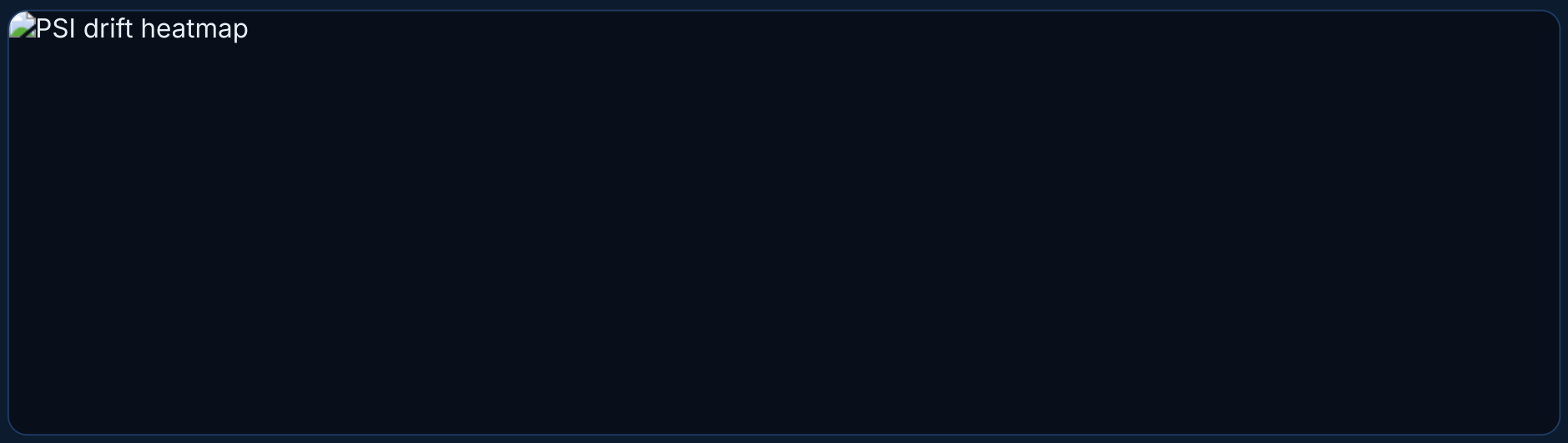
The fix: Changed `torch.float16` to `torch.bfloat16`. Removed GradScaler. AUC-PR recovered from 0.001 (collapsed) to 0.054, a 54x improvement. Hardware-aware numerical analysis for production training.

4	54x	bfloat16
Runs killed by float16	AUC-PR recovery	8-bit exponent fix

Why loss goes high, stays, then drops: Early epochs: model is random (high loss). Middle: learns easy majority first; FocalLoss then forces focus on hard minority (temporary loss bump). Late: cosine LR schedule stabilizes convergence.

40-DAY PSI DRIFT HEATMAP

Drift escalated for 13 days.
Then the retrain gate opened.



What is PSI? Population Stability Index measures distribution shift per feature per day. PSI < 0.1 = stable, 0.1-0.2 = warning, > 0.2 = critical. Calculated as $\sum(P_i - Q_i) \times \ln(P_i/Q_i)$ comparing production bins against training baseline. If any critical feature exceeds threshold, drift alarm fires.

D1-8 Baseline period. No drift. PSI near zero across all 6 features. Model performing at validation benchmarks.

D9 **False alarm:** retention_time_ms PSI spiked to 1.47 from a one-time batch anomaly (equipment calibration). Staleness gate prevented unnecessary retraining. Day 10 auto-recovered.

D11-29 Gradual drift escalated across features. PSI exceeded 5.0 by Day 29. Retrain blocked by staleness: model was <30 days old. AUC-PR degrading steadily.

D30 **RETRAIN TRIGGERED.** All 3 gates opened: PSI 5.25, AUC-PR dropped 8%, model 30 days old. 50 epochs on full day's data, bfloat16 on A100.

D31-35 Post-retrain recovery. New v2 model outperformed v1. AUC-PR improved above baseline.

D39 **CANARY FAILED.** Bad model deployed to 10% canary pod. AUC-PR crashed. **Automatic rollback to v2** within seconds.

D40 System recovered. Rolled-back v2 model stable. Pipeline proved self-healing without human intervention.

RETRAIN LOGIC

3-gate retrain trigger.
All 3 must be TRUE.

Gate 1: Statistical Drift (PSI)

Any critical feature PSI > 0.2 for 3+ consecutive days. Prevents reaction to single-day noise. retention_time_ms was the primary drift signal.

Threshold: PSI > 0.2, sustained 3+ days

Gate 2: Performance Degradation

>5% AUC-PR drop vs baseline. Compares current batch inference against the champion model's validation benchmark.

Threshold: 5% relative AUC-PR degradation

Gate 3: Model Staleness

>30 days since last retrain. Prevents reactive retraining from transient drift spikes that resolve on their own.

Threshold: 30 calendar days since last training

Result: Day 30 Retrain

All 3 gates opened simultaneously on Day 30. PSI at 5.25, AUC-PR dropped 8%, model was 30 days old. Fully automated retraining fired.

All 3 gates must be TRUE to retrain

Why 3 gates, not 1?

PSI alone fires on normal batch variance. Performance alone is noisy at 1:160 imbalance. Staleness alone wastes compute. Combined: precision retraining.

Low-Data Drift Tagging

When batch <10K rows, PSI is statistically unreliable due to sparse bin counts. Drift is **tagged in MLflow but never triggers retraining** until data volume is sufficient.

PRODUCTION INFRASTRUCTURE

6 services. One command. Fully orchestrated.

Apache Airflow

3 DAGs (Directed Acyclic Graphs): daily_inference, retrain_trigger, master orchestrator. Each DAG defines task dependencies without cycles. SLA monitoring alerts if any task exceeds its time budget. Automatic retry with exponential backoff.

Apache Kafka

Streaming DRAM test records from fab sensors. Consumer group routes records to FastAPI inference engine in real time. Handles 5M+ messages/day with partitioned topics for parallel processing.

Apache Spark

Batch ETL on 5M-row daily slices. 4.7x faster than pandas at this row count. Reads raw sensor data, applies feature engineering, writes parquet to S3. Total: 8.5GB parquet across 40 production days.

MLflow + Registry

Tracks every experiment run including the 4 failed float16 runs. Model Registry maintains champion/challenger versions. Automatic promotion on canary success, rollback on failure.

Prometheus + Grafana

Prometheus scrapes /metrics from FastAPI every 15 seconds. Grafana dashboards show per-feature PSI, inference latency (p50/p95/p99), request rates, prediction distribution. Alerts on SLA breach.

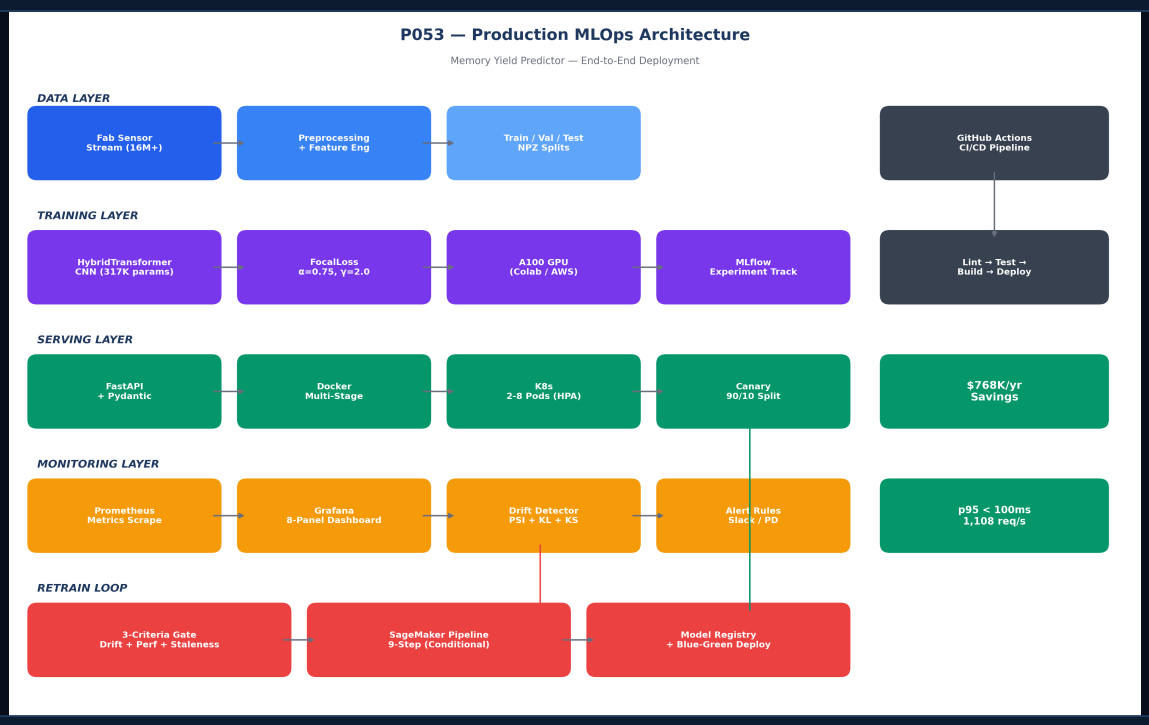
FastAPI + K8s

REST API with Pydantic validation, Redis prediction cache (5-min TTL). K8s with HPA (2-8 pods), canary deployment (90/10 traffic split). 120ms p99 latency under load testing.

To run locally: `cd deploy/docker && docker compose up -d` starts all 6 services. Airflow :8080, MLflow :5001, Grafana :3000, FastAPI :8000/docs.

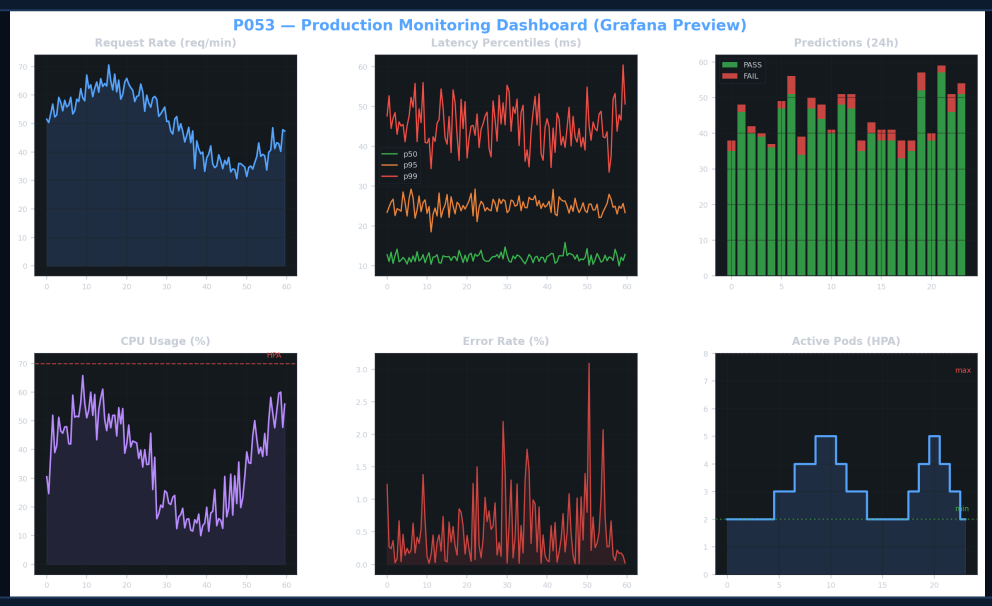
SYSTEM ARCHITECTURE

5-Layer MLOps:
Data, Train, Serve, Monitor, Retrain



Architecture: Data layer ingests 16M+ fab sensor records. Training layer runs HybridTransformerCNN with FocalLoss on A100 GPU. Serving layer delivers predictions via FastAPI in Docker/K8s. Monitoring layer tracks drift with PSI and model metrics via Prometheus/Grafana. Retrain loop fires only when all 3 gates are satisfied.

Production Monitoring Dashboard



Grafana shows: Request rate, latency percentiles (p50/p95/p99), prediction distribution (pass/fail ratio), CPU usage with HPA threshold, error rate, and pod autoscaling. All scraped by Prometheus every 15 seconds.

HARDWARE BENCHMARK

CPU training = 69 hours.
A100 = 1.6 hours. GPU is not optional.

Hardware benchmark

69 hr	3.5 hr	1.6 hr
CPU (Apple M3)	T4 (16GB)	A100 (40GB)

Wall clock time = actual elapsed time on your watch. The A100 processed 16M rows in 50 epochs in 1.6 hours of real time. The full 40-day production run (200M total rows, all drift checks, 1 retrain, canary evaluation, rollback logic) completed in 219 minutes wall clock on A100. Each "simulation day" represents one real production day in a DRAM fab, compressed into 5.5 minutes of compute.

Why HybridTransformerCNN on tabular data?

1D-CNN captures local patterns between adjacent features (e.g., leakage + retention combined signal). **Transformer** self-attention captures global feature interactions across all 6 features simultaneously. Per-feature tokenization gives each sensor its own learned embedding. This architecture consistently outperforms pure ML baselines on high-dimensional sensor data.

Data scale

16M training rows, 8.5GB total parquet across 40 production days (5M rows/day). All stored on S3 as compressed columnar parquet. Spark ETL handles daily batch processing. The pipeline is chip-agnostic: swap DRAM features for NAND flash or CPU wafer data, same architecture works.

40-DAY PRODUCTION RUN

Full system response to drift: detect, retrain, evaluate, rollback

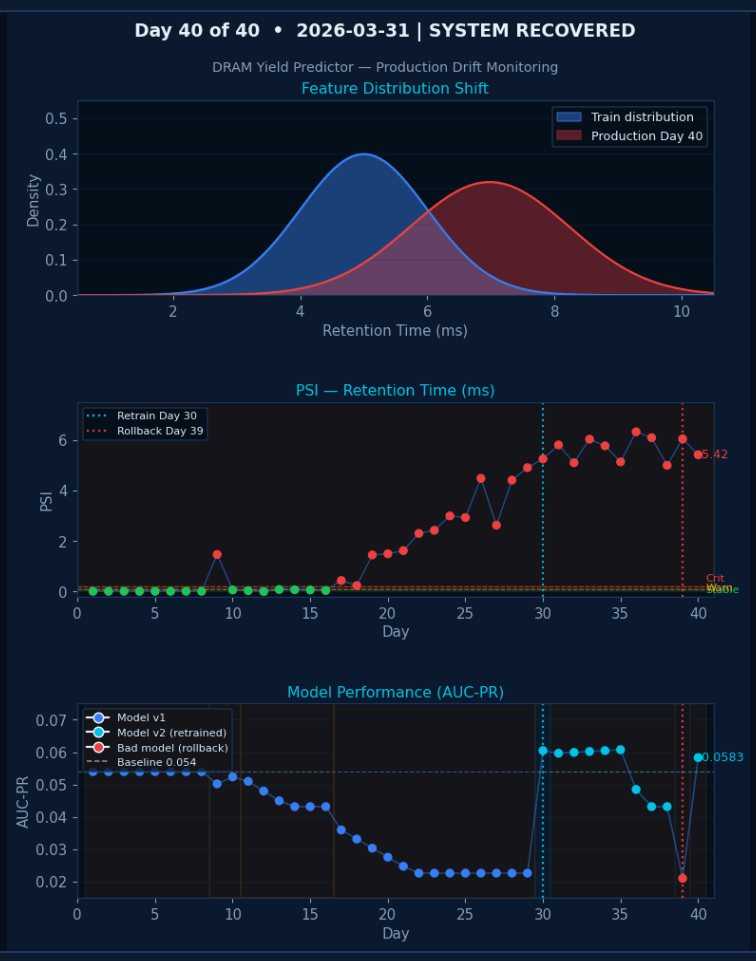
Simulation summary

40	200M	219 min	1
Production days	Total rows	Wall clock	Rollback

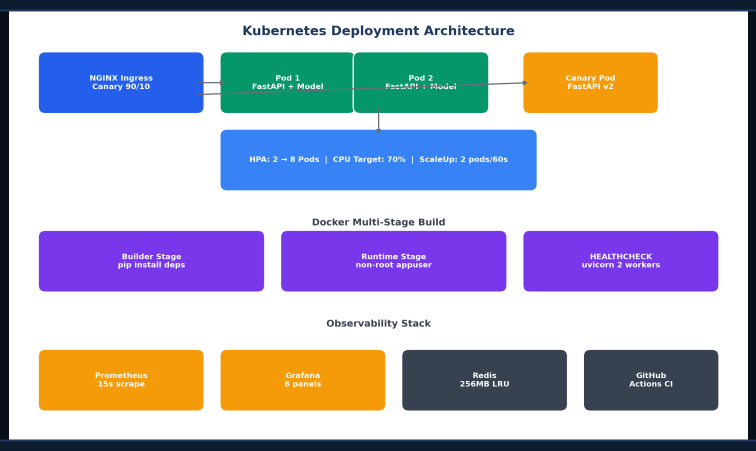
How to read this: 40 simulation days map to 40 real production days (Feb 20 to Mar 31, 2026). Each day processes 5M rows of DRAM sensor data. "Wall clock" = 219 minutes of actual GPU compute time to simulate the entire 40-day run. The system autonomously handled stable operation, gradual drift, a false alarm, triggered retraining, caught a bad model via canary testing, and auto-rolled back. Zero human intervention at any point.

FINAL STATE: DAY 40

System recovered.
Self-healing pipeline proven.



3-panel drift visualization (Day 40 of 40). Top: train vs production distribution for retention_time_ms showing clear shift. Middle: PSI per day with green/yellow/red severity, retrain (Day 30, cyan) and rollback (Day 39, red) markers. Bottom: AUC-PR showing v1 degradation, v2 recovery, canary failure, and final recovery. Full animated version (40 frames) available at `assets/drift_density_animation.gif`



github.com/rajendarmuddasani/DRAM_Yield_Predictor_MLOps